N O O T E R R A

System Architecture

Technical due diligence package.
Current implementation, target architecture, and research roadmap for the enterprise world model platform.

Confidential. Shared under NDA.
April 2026

Contents

1. System Overview 4
   1.1. Subsystem maturity 4
   1.2. Tech stack 4
2. Event Ledger 6
   2.1. Current implementation LIVE 6
   2.2. Target state TARGET 6
3. Canonical Object Graph 7
   3.1. Current implementation LIVE 7
   3.2. Target state TARGET 7
4. State Estimator and Belief System 8
   4.1. Current implementation LIVE 8
   4.2. Precise definitions 8
   4.3. Target state TARGET 9
5. World Model Ensemble 11
   5.1. Current implementation LIVE 11
   5.2. Model lifecycle 11
   5.3. Target state TARGET 12
6. Policy and Authority Engine 14
   6.1. Current implementation LIVE 14
   6.2. Target state TARGET 15
7. Action Gateway 16
   7.1. Current implementation LIVE 16
   7.2. Target state TARGET 16
8. Autonomy Engine 17
   8.1. Current implementation LIVE 17
   8.2. Target state TARGET 18
9. Simulation Layer 19
   9.1. Current implementation BUILDING 19
   9.2. Target state TARGET 19
10. Connector Framework 20
   10.1. Architecture 20
   10.2. Extraction pipeline 20
   10.3. Live connector: Stripe 20
11. Governance, Security, and Data Handling 21
   11.1. Tenant isolation 21
   11.2. Authentication 21
   11.3. Authorization 21

11.4. Audit trail 21
11.5. Incident response 21
11.6. Data handling 21
12. Operational Metrics and Testing 22
12.1. Test coverage 22
12.2. Failure modes 22
13. Technical Roadmap 23
14. References 24

1. System Overview

Nooterra is an enterprise world model platform. It ingests events from connected business systems, constructs a canonical representation of company state, estimates hidden operational variables, evaluates policy constraints, and governs agent actions through an evidence-based autonomy framework.

The platform is a hybrid system. The language model is a bounded reasoning component within a larger operational architecture. It is not the system of record, the policy authority, or the source of trust.

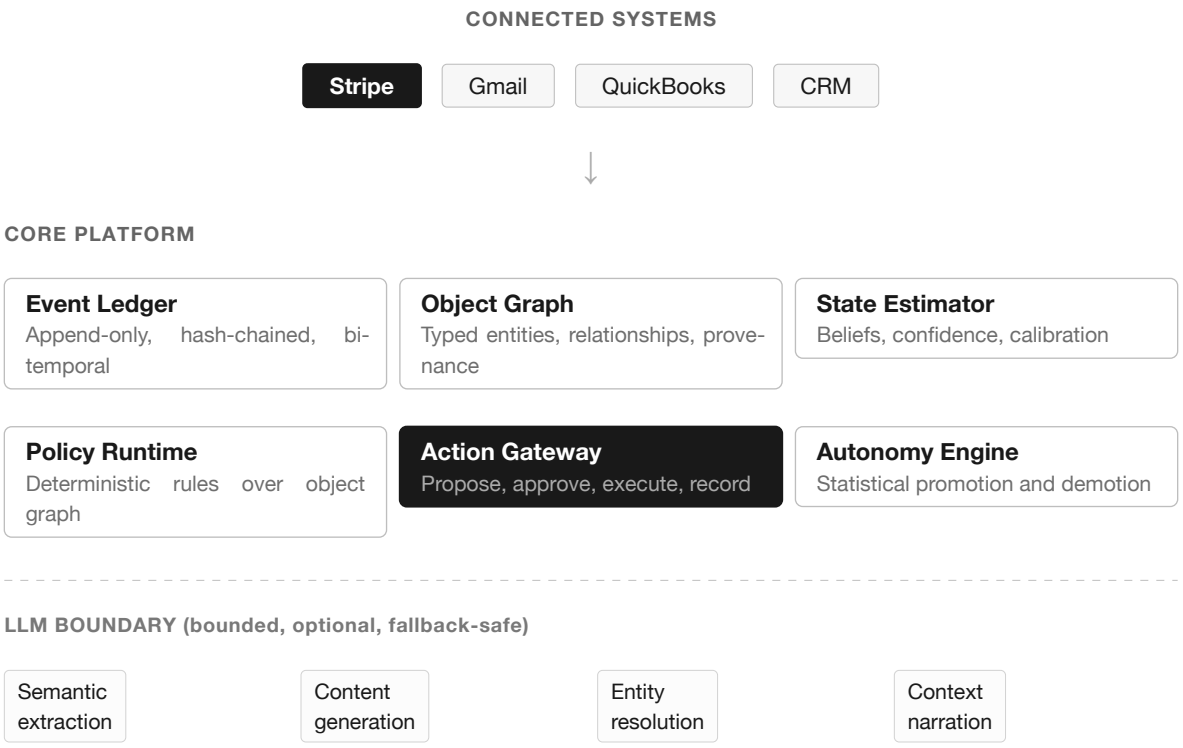
1.1. Subsystem maturity

Subsystem	Status	Details
Event Ledger	LIVE	Append-only, hash-chained, bi-temporal. PostgreSQL.
Object Graph	LIVE	5 canonical types. Typed relationships, version history, provenance.
State Estimator	LIVE	Rule-based + logistic regression. Confidence scoring. Conflict detection.
Policy Runtime	LIVE	Zanzibar-style DAG. Deterministic guards. LLM judgments with fallback.
Action Gateway	LIVE	11-step pipeline. Evidence bundles. Reversibility tracking.
Autonomy Engine	LIVE	Per-action-class trust levels. Procedural + outcome scoring. Asymmetric demotion.
Stripe Connector	LIVE	Webhook ingestion. Typed WorldEvents. 5 canonical object types.
Multi-Source Connectors	BUILDING	Gmail, QuickBooks, CRM. Framework in place, connectors in development.
Simulation Layer	BUILDING	Rule-based scenario analysis. Causal simulation in design.
Conformal Prediction	TARGET	MAPIE integration for statistically guaranteed prediction intervals.
Causal Inference	TARGET	DoWhy + EconML for treatment effect estimation. CausalNex for simulation.
Inter-Company Network	TARGET	Agent identity, capability discovery, negotiation protocol.

1.2. Tech stack

- **Runtime:** TypeScript (strict mode), Node.js
- **Database:** PostgreSQL (Railway), Redis (optional caching)
- **LLM routing:** OpenRouter (OpenAI, Anthropic, Google)
- **Integrations:** Composio (250+ app connectors), Stripe API (direct)
- **Infrastructure:** Railway (production), Docker Compose (local), Helm charts (Kubernetes)
- **Auth:** Magic link (email OTP) + passkeys. No password storage.
- **Testing:** 253 tests across unit, integration, contract, chaos, and load categories. 0 failures.

FIGURE 1: PLATFORM ARCHITECTURE



2. Event Ledger

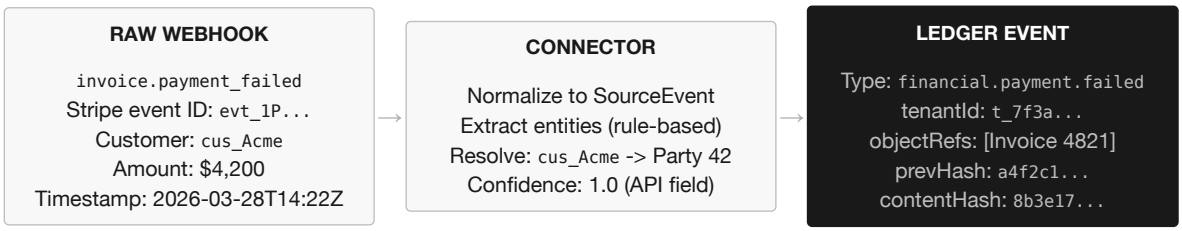
The event ledger is the audit foundation of the system. Every observation, action, and state change is recorded as an immutable, append-only event.

2.1. Current implementation LIVE

Events are stored in a PostgreSQL table partitioned by tenant and time. Each event carries:

- **Type:** A hierarchical event type (e.g., `financial.invoice.created`, `financial.payment.received`, `relationship.party.created`)
- **Timestamps:** Bi-temporal. `timestamp` records when the event happened in the real world. `recordedAt` records when the system learned about it. This distinction is critical for late-arriving data: an invoice dated March 1 but received April 15 must be recorded with both times.
- **Hash chain:** Each event stores a content hash and the previous event's hash per tenant. Chain breaks are detectable. This provides tamper-evidence without requiring blockchain infrastructure.
- **Provenance:** `sourceSystem` (stripe, gmail, etc.), `sourceId` (ID in the original system), `extractionMethod` (api, llm, rule, human), `extractionConfidence` (0 to 1), `rawRef` (reference to the raw source event before extraction).
- **Immutability:** Events are never mutated. Corrections create new events with a `causedBy` reference to the original. The system can reconstruct any historical state by replaying events up to a given timestamp.

FIGURE 2: EVENT FLOW FROM STRIPE WEBHOOK TO LEDGER



2.2. Target state TARGET

- **Full bi-temporal queries:** "What did the system believe about Customer X on March 15?" requires querying both assertion time (when the belief was current) and transaction time (when the system recorded it). The schema supports this; the query interface is in development.
- **Content-addressed event references:** Cross-tenant audit without data sharing. A tenant can prove an event existed at a given time by providing its content hash without revealing the event contents.

3. Canonical Object Graph

The object graph is the core representation of company state. It transforms raw source records into canonical business objects with typed relationships, version history, and provenance tracking.

3.1. Current implementation LIVE

Five canonical object types for the initial AR domain:

Object Type	Description	Source
Party	Customer or counterparty. Name, email, identifiers, metadata.	Stripe
Invoice	Billing document. Amount, status, due date, line items.	Stripe
Payment	Money movement. Amount, method, settlement status.	Stripe
Dispute	Chargeback or complaint. Reason, evidence deadline, status.	Stripe
Conversation	Communication thread. Messages, participants, sentiment.	Gmail (target)

Each object carries:

- **Version history:** Every mutation creates a new version. Previous versions are retained. The system can reconstruct object state at any historical point.
- **Provenance per field:** Each field records which source system provided the value, with extraction confidence. When multiple sources report the same field, the system tracks all contributions.
- **Temporal validity:** `valid_from` and `valid_to` timestamps for temporal queries.
- **Relationships:** Typed edges between objects (e.g., Party OWES Invoice, Payment SETTLES Invoice, Dispute CHALLENGES Payment). Relationships are first-class objects with their own version history.
- **Entity resolution:** When multiple sources reference the same real-world entity (e.g., “Acme Corp” in Stripe and “ACME Corporation” in QuickBooks), the system resolves them to a single canonical object with provenance from both sources.

3.2. Target state TARGET

- **20+ object types:** Deal, Contract, Obligation, Task, ScheduleSlot, Document, and others as domain packs expand.
- **Temporal knowledge graph embeddings:** TTransE and TNTComplEx embed entities, relations, and timestamps into the same vector space. This enables link prediction: “Based on temporal patterns, Customer X is likely to churn within 30 days” or “Vendor Y is likely to submit an invoice next week.” These are research-stage techniques (see Cai et al., 2024 for a survey of temporal KG methods).
- **Full bi-temporality:** Assertion time (when the fact was true in reality) plus transaction time (when the system recorded it), enabling historical replay for audit and debugging.

4. State Estimator and Belief System

Not every important business fact is directly observable. The state estimator maintains an estimated state layer that separates observed facts from inferred beliefs.

4.1. Current implementation LIVE

The core abstraction is the **Belief**: an estimated fact with explicit uncertainty.

```
interface Belief {
  field: string
  value: unknown
  confidence: number   sources: SourceContribution[]
  estimatedAt: Date
  method: 'direct_observation' | 'inference'
    | 'aggregation' | 'default'
}
```

Four estimation methods, applied in order of preference:

1. **Direct observation aggregation.** Multiple sources report the same field. The system uses recency and source reliability weighting. A Stripe API field (confidence 1.0) outweighs an LLM-extracted email field (confidence 0.7).
2. **Rule-based inference.** Deterministic rules derive beliefs from observable data. Example: invoice overdue 30+ days with no response to 2 emails implies payment probability drops below 0.4.
3. **Learned inference.** Statistical models trained on historical data. Logistic regression and XGBoost for payment propensity, churn risk, response likelihood, and dispute probability. Each model tracked independently with its own calibration history.
4. **Default priors.** When no data exists for a specific entity, the system uses industry and segment priors. These are explicitly labeled as low-confidence defaults.

Conflict detection. When multiple sources report different values for the same field, the system creates a first-class Conflict object. Simple field conflicts auto-resolve (newer observation wins). Complex conflicts surface for human resolution. All conflicts are logged for audit.

Calibration tracking. The CalibrationTracker computes MAE (Mean Absolute Error) per model per prediction target. Every prediction is stored alongside its eventual outcome. Calibration reports compare predicted probabilities against observed frequencies.

4.2. Precise definitions

These terms are used throughout the system and are defined here for auditability:

- **Confidence:** The model's estimate of its own certainty for a specific prediction (0 to 1). A confidence of 0.85 means the model estimates an 85% chance its prediction is correct.
- **Calibration:** How well historical confidence estimates matched actual outcomes. A well-calibrated model with confidence 0.70 should see approximately 70% of those predictions come true. Measured as 1 minus MAE between predicted probabilities and observed frequencies.

- **Belief:** An estimated fact with a value, confidence, provenance chain (which sources contributed), estimation method, and timestamp. Beliefs are mutable; they update as new evidence arrives. Historical beliefs are preserved via version history.
- **Conflict:** When two or more sources report different values for the same field on the same object. Conflicts are first-class objects with resolution metadata.

4.3. Target state TARGET

4.3.1. Conformal prediction (MAPIE)

The current system outputs point estimates with scalar confidence. The target state uses conformal prediction via the MAPIE library to produce statistically guaranteed prediction intervals.

Instead of “payment probability: 0.72, confidence: 0.6”, the system will output “payment probability: 0.72, 90% prediction interval: [0.58, 0.86].” The interval carries a finite-sample coverage guarantee: regardless of the underlying model quality, the true value will fall within the interval at least 90% of the time.

For time series predictions (days to payment, DSO forecasts), the system will use the EnbPI (Ensemble batch Prediction Intervals) variant, which drops the exchangeability requirement that standard conformal methods assume.

Implementation path: MAPIE is pip-installable and wraps existing sklearn-compatible models. A Python sidecar service receives prediction requests and returns intervals. This is deployable in weeks.

4.3.2. Post-hoc calibration

Two complementary approaches:

- **Temperature scaling** (Guo et al., 2017): a single learned parameter T that rescales model outputs before softmax. Surprisingly effective on overconfident models. Fit by minimizing negative log-likelihood on a held-out calibration set.
- **Isotonic regression:** a non-parametric calibration method that fits a monotonic step function. More flexible than Platt scaling. Available in scikit-learn.

The system will use whichever method produces lower Expected Calibration Error (ECE) on holdout data for each prediction target.

4.3.3. Drift detection (ADWIN)

ADWIN (ADaptive WINdowing), via the River library for online machine learning, monitors prediction residuals (predicted minus actual) per model per tenant. It maintains a variable-length window and detects statistically significant distribution changes. When drift is detected, the system:

1. Flags the affected model for review
2. Widens prediction intervals automatically
3. Triggers recalibration or retraining if drift persists

Implementation: one class from the River library, approximately 5 lines of integration code per model. Runs as a streaming monitor on the prediction-outcome pair stream.

4.3.4. Out-of-distribution detection

Feature distribution monitoring via KL divergence. The system tracks the distribution of input features at training time. At inference time, it computes KL divergence between the current input distribution and the training distribution. When divergence exceeds a threshold, the system:

1. Flags predictions as low-confidence regardless of model output
2. Alerts the operator that the model is operating outside its training distribution
3. Falls back to more conservative (wider interval, lower autonomy) behavior

Conformal p-values provide a second OOD signal for free from the MAPIE infrastructure: data points with nonconformity scores above the calibration quantile are flagged as potentially out-of-distribution.

5. World Model Ensemble

The world model is not a single monolithic model. Each prediction target gets its own model instance. Models are independent, testable, and replaceable. The ensemble combines four model types, deployed in a progression based on data availability and calibration evidence.

5.1. Current implementation LIVE

5.1.1. Model Type 1: Deterministic rules

Accounting identities, contract terms, deadline calculations, permission checks, budget arithmetic. These are always correct when their preconditions hold.

Example: “If invoice.dueAt < now AND invoice.status = ‘sent’, then invoice.status = ‘overdue’.”

5.1.2. Model Type 2: Probabilistic sequence models

Logistic regression and XGBoost trained on historical event sequences per object type. Prediction targets for the initial AR domain:

Target	Type	Horizon
Payment probability	Float [0,1]	7, 30, 90 days
Days to payment	Distribution	Per invoice
Churn risk	Float [0,1]	30, 90 days
Dispute probability	Float [0,1]	Per invoice
Customer lifetime value	Distribution	12 months
DSO forecast	Distribution	30/60/90 days

Each prediction is logged as a `system.model.prediction` event in the ledger, stored alongside its eventual outcome for calibration.

5.1.3. Model Type 3: Intervention estimates (current: heuristic)

The current implementation uses hardcoded heuristics to estimate the effect of actions. For example, sending a collection email is estimated to improve payment probability by 0.15. These are initial approximations that will be replaced by causal inference (see target state below).

5.1.4. Model Type 4: Scenario analysis (current: rule-based)

Rule-based scenario analysis for simple what-if questions. “What happens to DSO if we shorten payment terms from 30 to 15 days?” Answered by applying deterministic rules forward from the proposed change.

5.2. Model lifecycle

1. **Training data:** Historical events from the ledger, joined with observed outcomes
2. **Feature engineering:** Object graph attributes, temporal features (days since last contact, payment velocity), relationship features (open tickets, renewal proximity)
3. **Training:** Standard sklearn/XGBoost pipeline
4. **Calibration testing:** ECE on holdout set. Models with ECE above threshold are rejected.
5. **Shadow deployment:** Model runs in parallel with production. Predictions logged but not used for decisions. Shadow period: minimum 14 days.

6. **Live deployment:** Model replaces the previous version. Continuous calibration monitoring.
7. **Retraining triggers:** ADWIN drift detection (target state), calibration degradation, or manual review.

5.3. Target state TARGET

5.3.1. Causal inference via DoWhy + EconML

The highest-leverage upgrade to the world model. Replaces hardcoded intervention heuristics with Conditional Average Treatment Effects (CATE).

DoWhy (Microsoft/PyWhy) provides a 4-step causal inference API:

1. **Model:** Specify the causal graph (DAG) relating actions, confounders, and outcomes
2. **Identify:** Find the valid adjustment set for the desired causal effect
3. **Estimate:** Compute the effect using Double ML, causal forests, or instrumental variables
4. **Refute:** Test sensitivity to assumptions (placebo treatment, random common cause, data subset)

EconML provides the estimators. Causal forests (Athey & Wager, 2018) estimate heterogeneous treatment effects: different customers respond differently to the same action. Instead of "sending a collection email improves payment probability by 0.15" (a global average that may be wrong for any individual customer), the system will output "sending a collection email to THIS customer with THIS payment history increases payment probability by 0.22 +/- 0.08."

Implementation: Python sidecar service. Ingests the `world_predictions` and `world_prediction_outcomes` tables. Outputs per-customer CATE estimates with confidence intervals. The existing `estimateIntervention()` function becomes a thin client to this service.

5.3.2. Causal discovery via NOTEARS + PC algorithm

Before estimating causal effects, the system needs a causal graph. Two approaches:

- **PC algorithm:** Uses conditional independence tests to learn the DAG structure from observational data. Fast, well-understood, available in the `causal-learn` library.
- **NOTEARS:** Reformulates DAG learning as a continuous optimization problem with an acyclicity constraint. More scalable than PC but has known issues with scale-invariance on dimensional quantities (see Kaiser & Sipos, 2025).

The system will use PC for initial structure learning, validate with domain experts, and refine incrementally as more data accumulates.

5.3.3. Simulation via CausalNex

CausalNex (McKinsey/QuantumBlack) builds Bayesian networks from data and expert knowledge, then supports do-calculus interventions.

Replace: `Math.min(1, currentPayProb + 0.15)`
With: `bn.do(intervention=\{'sent_email': 1\}).query('payment_7d')`
Returns: an actual posterior distribution, not a magic number.

5.3.4. Monte Carlo rollout with causal constraints

Once the causal DAG is in place, the system can sample from the joint distribution implied by the graph, respecting causal ordering. Interventions propagate forward through time. Output: "If we change collections cadence, here are 1,000 simulated futures with confidence bands on revenue, churn, and support load."

This is the full realization of the simulation layer: scenario cards with predicted impact, confidence intervals, tradeoffs, key assumptions, and known unknowns.

6. Policy and Authority Engine

The policy engine decides what is allowed. It evaluates deterministic rules over the canonical object graph. It is separate from the LLM reasoning layer by design.

6.1. Current implementation LIVE

6.1.1. Authority graph

A directed acyclic graph (DAG) modeled after Google's Zanzibar authorization system. Humans grant authority to agents. Agents can delegate to sub-agents. Every edge is an AuthorityGrant with explicit scope:

- **Action classes:** Which types of actions can be taken (e.g., `send_collection_email`, `issue_refund`)
- **Object types:** Which object types can be acted upon
- **Party filters:** Which counterparties (e.g., "only customers with LTV > $10K")
- **Budget limits:** Maximum value per action and per time window
- **Time windows:** When actions are permitted
- **Jurisdiction:** Geographic or regulatory scope

Authority only attenuates. A child's authority is always a subset of its parent's. This is enforced structurally: the grant evaluation function intersects the parent's scope with the child's requested scope.

6.1.2. Policy guards

Deterministic, fast, auditable rules that evaluate to allow, deny, or require_approval.

```
interface PolicyGuard {
  id: string
  description: string
  condition: PolicyPredicate
  effect: 'allow' | 'deny' | 'require_approval'
  priority: number
}
```

Guards are evaluated in priority order. The first matching guard determines the outcome. If no guard matches, the system defaults to `require_approval`.

6.1.3. Policy judgments

For ambiguous cases where deterministic rules are insufficient, the system evaluates an LLM-based judgment. Each judgment has a trigger condition, an evaluation prompt, a fallback decision (used if the LLM times out or fails), and a timeout.

Judgments are secondary to guards. Guards are always evaluated first. Judgments are only reached when no guard produces a definitive answer.

6.1.4. neverDo lists

Actions that are categorically forbidden. These lists are immutable from autonomous code paths. Only human operators can modify them. Examples: "Never issue a refund exceeding the original invoice amount." "Never contact a customer flagged as legally represented."

6.1.5. Authorization decision flow

FIGURE 3: AUTHORIZATION PIPELINE

1. Extract from proposed action: action class, target objects, counterparties, value, timing
2. Load agent's effective authority (all grants merged via intersection)
3. Check neverDo list. DENY if match.
4. Check scope: object type, party filter, budget, jurisdiction, time. DENY if out of scope.
5. Evaluate deterministic guards in priority order. ALLOW / DENY / REQUIRE_APPROVAL.
6. If no guard matched: evaluate LLM-based judgments (with fallback and timeout).
7. If still no decision: default to REQUIRE_APPROVAL.
8. If ALLOW: check rate limits and budget (atomic). Execute. If REQUIRE_APPROVAL: queue for human.

Every decision is logged with full reasoning chain for audit.

6.2. Target state TARGET

6.2.1. LLM-to-formal-policy compilation

The PolicyEditor UI lets humans write policies in natural language. The target state compiles these into executable symbolic constraints with soundness verification. Recent work on neurosymbolic verification achieves 99%+ soundness on policy-to-formal translations (see Cosler et al., 2023).

Example: "Never send more than 2 collection emails per week per customer" compiles to a deterministic guard with a rate-limit predicate and per-customer, per-week windowing.

6.2.2. Neurosymbolic guard evaluation

Every LLM-proposed action passes through a symbolic constraint checker that evaluates the action against the object graph invariants and the policy engine. This catches structurally invalid proposals ("refund larger than invoice," "contact legally represented customer") before they reach the gateway.

7. Action Gateway

The gateway is the single chokepoint for all side effects. Every action that touches the external world passes through an 11-step pipeline.

7.1. Current implementation LIVE

Step	Name	Function
1	**Authenticate**	Verify agent identity and authority grant chain
2	**Authorize**	Evaluate against policy and authority engine (Section 6)
3	**Validate**	Check parameters, counterparty, value limits
4	**Rate Limit**	Enforce action frequency limits per class per tenant
5	**Budget Check**	Verify budget available, atomic decrement
6	**Disclosure**	Inject required disclosures (e.g., “AI composed this message”)
7	**Simulate**	Optional. World model predicts downstream effects. Escalate if negative.
8	**Escrow**	Low risk: execute. Medium: hold. High: hold and notify human.
9	**Execute**	Call tool or integration. Capture result. Record compensating action.
10	**Audit**	Write event to ledger. Generate evidence bundle. Update object graph.
11	**Notify**	Trigger state estimator, evaluator, and planner updates.

7.1.1. Evidence bundles

Every executed action produces a mandatory evidence bundle containing: the proposed action, the policy predicates evaluated, the facts relied on (with object IDs), the tools used, the agent’s self-assessed uncertainty, the approval decision (human or automatic), the execution result, the reversible path (how to undo), and the authority chain.

Evidence bundles are stored in the append-only event ledger with hash-chain integrity. They are immutable and retained for the lifetime of the tenant account.

7.1.2. Reversibility tracking

Each action class is annotated with reversibility metadata: full (can be completely undone), partial (some effects are reversible), or none (irreversible). Compensating actions are defined per action class. Time windows for rollback are tracked. The escrow decision in step 8 weighs reversibility: irreversible actions face a higher bar for autonomous execution.

7.2. Target state TARGET

- **Pre-execution simulation via causal digital twin:** Step 7 (Simulate) currently uses rule-based heuristics. The target state connects to the CausalNex Bayesian network for actual posterior predictions of downstream effects.
- **Intent-based multi-agent coordination:** Before an agent acts on an object, it declares intent (“I plan to send a collection email to Invoice X”). A coordinator checks for conflicts (another agent plans to call the customer about the same invoice). Uses optimistic concurrency with intent TTLs.

8. Autonomy Engine

Autonomy is not assigned globally. It is earned per action class from evidence. New action classes begin supervised. Promotion depends on repeated successful execution, procedural quality, business outcomes, and incident frequency. Demotion is immediate.

8.1. Current implementation LIVE

8.1.1. Trust levels

Level	Behavior	Promotion Criteria
Locked	Action type forbidden	Explicit policy enablement
Shadow	System proposes, no user visibility	Internal testing only
Supervised	User approves each action	Default for all new action types
Auto + Review	Executes, user reviews after	See thresholds below
Autonomous	No human involvement	See thresholds below

8.1.2. Scoring

Procedural score. Did the action follow the policy template? Measured as the percentage of policy predicates that evaluated to true at execution time. Binary per-predicate, averaged across the action.

Outcome score. Did the predicted result match the observed result? Measured as whether the actual outcome fell within the model's predicted confidence interval. Averaged over the evaluation window.

Incident. Any of: (a) a customer complaint attributed to the action, (b) a constraint violation detected post-execution, or (c) an observed outcome outside 2-sigma of the model's prediction. Classified as minor (recoverable, no customer impact), moderate (customer-visible, recoverable), or critical (financial loss, relationship damage, or compliance violation).

8.1.3. Promotion thresholds

Supervised to Auto + Review. Rolling 30-day window, per action class, per tenant:

- At least 20 completed executions
- Procedural score at or above 85%
- Outcome score at or above 75%
- At most 1 minor incident, 0 moderate or critical

Auto + Review to Autonomous. Rolling 60-day window:

- At least 50 completed executions
- Procedural score at or above 90%
- Outcome score at or above 80%
- Zero incidents of any severity

8.1.4. Asymmetric demotion

A single moderate or critical incident triggers immediate demotion to Supervised for the affected action class. The system notifies the operator, provides the full evidence bundle, and resets the trust score.

8.1.5. Sparse-data safeguards

In low-volume environments where 20 or 50 executions take months to accumulate, the system does not promote. There is no interpolation, no borrowed confidence from other tenants, and no default promotion schedule.

8.2. Target state TARGET

8.2.1. Active learning via Bayesian acquisition functions

Instead of “propose the action with highest expected value,” propose “the action whose outcome will teach us the most.” Expected Information Gain as the acquisition function. Early in a tenant’s lifecycle, this accelerates calibration dramatically by focusing human review on the most informative cases.

8.2.2. BALD (Bayesian Active Learning by Disagreement)

Select predictions where ensemble members disagree most. Maximizes mutual information between model parameters and predictions. When ensemble models disagree on a prediction, that is exactly where human input is most valuable. This focuses operator attention where it has the highest return.

9. Simulation Layer

The long-term purpose of the world model is to simulate what different actions are likely to produce before they occur.

9.1. Current implementation BUILDING

Rule-based scenario analysis and hardcoded intervention estimates. The `estimateIntervention()` function applies fixed deltas to current state estimates. Example: sending a collection email increases payment probability by 0.15. These are initial approximations, explicitly acknowledged as heuristic.

The simulation layer is the most aspirational subsystem. The current state is heuristic. The target state is causal. The path from here to there requires sufficient action-outcome data density per tenant.

9.2. Target state TARGET

9.2.1. CausalNex Bayesian network

Build a Bayesian network from data and expert knowledge. Support do-calculus interventions that return actual posterior distributions. Replace hardcoded deltas with learned causal effects.

9.2.2. Twin networks for counterfactuals

Pearl's twin network construction duplicates every endogenous variable to compute "what would have happened under a different action." This enables true counterfactual reasoning: "Would this invoice have been paid if we had sent the email last week?" Requires full SCM specification, which requires more data density than most tenants will have in early phases.

9.2.3. Monte Carlo simulation with causal constraints

Sample from the joint distribution implied by the causal DAG, respecting causal ordering. Propagate interventions forward through time. Output: scenario cards with predicted impact (confidence intervals), tradeoffs, key assumptions, and known unknowns.

9.2.4. Honest assessment

Causal simulation at enterprise scale is an open research problem. The techniques above are well-established in academic settings and in specific domains (epidemiology, economics). Applying them to multi-system enterprise operations with sparse, noisy data per tenant is harder. The system design accommodates this by falling back to heuristics when data is insufficient for causal methods. The progression from rules to statistics to causal models is data-driven, not calendar-driven.

10. Connector Framework

Connectors are the observation layer. They ingest events from external systems and transform them into typed WorldEvents for the ledger.

10.1. Architecture

Each connector implements a standard interface:

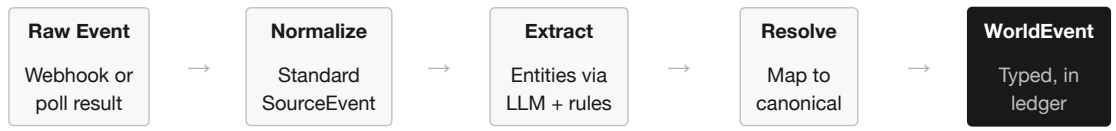
```
interface Connector {
  type: ConnectorType
  tenantId: string
  credentials: EncryptedCredentials
  syncCursor: string
  status: ‘active’ | ‘paused’ | ‘error’
  lastSyncAt: Date

  poll(): AsyncIterable<SourceEvent>
  handleWebhook(payload: unknown): SourceEvent[]
}
```

Connectors are stateless processors. State lives in the sync cursor (a resumption point for incremental ingestion). This means connectors can be restarted, scaled, or replaced without data loss.

10.2. Extraction pipeline

FIGURE 4: EXTRACTION PIPELINE



- **Structured sources** (Stripe, QuickBooks API): Rule-based extraction. Confidence 1.0 for direct API fields.
- **Unstructured sources** (email bodies, transcripts): LLM-based extraction with confidence scoring. Results stored with `extractionMethod: 'llm'` and the confidence value.
- Every raw event is written to the ledger before downstream processing, even if extraction fails. This ensures no observation is lost.

10.3. Live connector: Stripe

Webhook events ingested: `invoice.created`, `invoice.paid`, `invoice.payment_failed`, `payment_intent.succeeded`, `charge.dispute.created`, `customer.created`, `customer.updated`.

These transform into typed WorldEvents: `financial.invoice.created`, `financial.payment.received`, `financial.dispute.opened`, `relationship.party.created`.

Object graph types populated: Party, Invoice, Payment, Dispute.

11. Governance, Security, and Data Handling

11.1. Tenant isolation
Every tenant operates in a fully isolated data partition. Event ledger, object graph, state estimates, policy rules, and autonomy scores are partitioned by tenant ID at the database level. Cross-tenant queries are architecturally impossible: the query layer enforces tenant scoping at the connection level. There is no shared state between tenants.

11.2. Authentication
User authentication through magic link (email OTP) and passkey-based (WebAuthn) flows. No passwords are stored. Session tokens are scoped to the tenant and carry explicit expiration.

11.3. Authorization
The Zanzibar-style authority graph (Section 6) governs all action execution. Approval authority is explicit: only users with the appropriate grant can approve actions in a given action class. The authority graph is itself auditable: all grant changes are logged in the event ledger.

11.4. Audit trail
All evidence bundles (Section 7) are stored in the append-only event ledger with hash-chain integrity. They are immutable and retained for the lifetime of the tenant account. The system can reconstruct the complete decision history for any action: what was proposed, what facts were considered, what policy was applied, who approved, what happened, and what the outcome was.

11.5. Incident response
When the autonomy engine detects an incident:
1. Demote the affected action class to Supervised (immediate)
2. Notify the human operator with the full evidence bundle
3. Log the incident with severity classification
4. For critical incidents: pause all autonomous actions for the tenant until human review

11.6. Data handling
- **PII:** Customer names, email addresses, and other PII are stored as received from the source system. Subject to tenant data retention configuration.
- **Credentials:** No raw credentials stored beyond OAuth tokens managed through the connector framework.
- **Deletion:** Tenant data deletion is supported. Removing a tenant removes all associated events, objects, estimates, and policy state.
- **Encryption:** Data at rest encrypted via PostgreSQL transparent data encryption. Data in transit over TLS.

12. Operational Metrics and Testing

12.1. Test coverage

253 tests across 5 categories, 0 failures:

Category	Coverage	Count
Unit	Core logic, object graph operations, policy evaluation, belief computation	120
Integration	Connector ingestion, gateway pipeline, autonomy promotion, billing	60
Contract	API surface stability, webhook schema validation	30
Chaos	Connector failure, latency injection, malformed webhooks, partial outages	20
Load	100 concurrent tenants, 10,000 objects each	23

12.2. Failure modes

- **Connector failure:** System continues with stale data. Alerts operator. Predictions widen confidence intervals.
- **LLM unavailable:** Template-based fallbacks for all LLM functions. System continues operating.
- **Database degradation:** Read replicas for query load. Write failures halt action execution (fail safe).
- **Model degradation:** ADWIN drift detection (target state) triggers automatic fallback to conservative estimates and wider intervals.

The design principle: degrade gracefully, fail safe, never fail silent.

13. Technical Roadmap

Priority stack for state-of-the-art techniques, ordered by impact relative to implementation effort:

#	Technique	Library	Timeline
1	Conformal prediction for guaranteed intervals	MAPIE	Weeks
2	Drift detection on prediction residuals	River (ADWIN)	Weeks
3	Post-hoc calibration (temperature scaling + isotonic)	scikit-learn	Weeks
4	Feature distribution monitoring for OOD detection	Custom (KL div)	Weeks
5	LLM-to-formal-policy compilation	Custom + LLM	1-2 months
6	Intent-based multi-agent coordination	Custom	1-2 months
7	Causal effect estimation (CATE)	DoWhy + EconML	2-3 months
8	Bayesian network simulation	CausalNex	2-3 months
9	Full bi-temporal object graph	PostgreSQL	1-2 months
10	Active learning acquisition functions	Custom	3-4 months

Items 1-4 are Python services that can be deployed independently. Items 5-6 are TypeScript extensions to the existing codebase. Items 7-10 require more data infrastructure and depend on sufficient action-outcome data density.

14. References

1. Gupta, Li, Liu, Subramanian, Suleman, Zhang, Lu, Pasupalak. “World of Workflows: A Benchmark for Bringing World Models to Enterprise Systems.” arXiv:2601.22130. January 2026.
2. Guo, Pleiss, Sun, Weinberger. “On Calibration of Modern Neural Networks.” ICML 2017. (Temperature scaling for post-hoc calibration.)
3. Athey, Wager. “Estimation and Inference of Heterogeneous Treatment Effects using Random Forests.” Journal of the American Statistical Association, 2018. (Causal forests for CATE estimation.)
4. Sharma, Kiciman. “DoWhy: An End-to-End Library for Causal Inference.” 2020. (4-step causal inference API.)
5. Beaumont, Sheridan, et al. “CausalNex: Toolkit for Causal Reasoning with Bayesian Networks.” McKinsey/QuantumBlack. (Bayesian network do-calculus.)
6. Romano, Patterson, Candes. “Conformalized Quantile Regression.” NeurIPS 2019. (Conformal prediction for distribution-free intervals.)
7. Taquet, Blot, Morzadec, Lacombe, Brunel. “MAPIE: Model Agnostic Prediction Interval Estimator.” JMLR 2022.
8. Bifet, Gavalda. “Learning from Time-Changing Data with Adaptive Windowing.” SDM 2007. (ADWIN algorithm for drift detection.)
9. Cai, Liang, Wang, et al. “Temporal Knowledge Graph Completion: A Survey from the Perspective of Representation Learning.” Applied Sciences, 2024.
10. Pearl. “Causality: Models, Reasoning, and Inference.” Cambridge University Press, 2009. (Structural causal models, do-calculus, twin networks.)
11. Kaiser, Sipos. “On the Limitations of NOTEARS for Causal Discovery.” 2025. (Scale-invariance issues with continuous DAG learning.)
12. Zanzibar: Google’s Consistent, Global Authorization System. ATC 2019. (Inspiration for the authority graph design.)

N O O T E R R A

System Architecture
nooterra.ai

April 2026. Confidential. Shared under NDA.